IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2021

DIGITAL ELECTRONICS & COMPUTER ARCHITECTURE

Friday, 30 April 10:00 am

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions. Question One carries 40 marks. Question Two and Question Three carry 30 marks each.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : T.J.W. Clarke, Z. Durrani
Second Marker(s) : J. Wickerson, M. Dai

© Imperial College London

Special instructions for invigilators: None

Information for candidates:

In the figures showing digital circuits, all components have, unless explicitly indicated otherwise, been drawn with their inputs on the left and their outputs on the right. All signals labelled with the same name are connected together. All circuits use positive logic. The least significant bit of a bus signal is labelled as bit 0, and the most significant bit with the highest integer number. Therefore the signal X(7:0) is an eight bit bus with X7 being the MSB and X0 the LSB. Furthermore, if A and B are registers, the notation A:B designates the registers concatenated with A more significant than B.

In questions involving circuit design, you may use any standard digital circuits that are not explicitly forbidden by the question provided that you fully specify their operation.

Marks may be deducted for unnecessarily complex designs unless you are explicitly instructed not to simplify your solution.

1. a) Prove using Boolean algebra:

$$A + \bar{A}B = A + B$$

[4]

b) Simplify the following expressions using Boolean algebra and De Morgan's theorem.

i) $B + (C \oplus \bar{B})(AB + \bar{C})$

[4]

ii) $\overline{AB + AC} + A\,\overline{B}\,C$

[4]

c) Use a Karnaugh map to minimise the following Boolean function in SOP format. Hence, by using Boolean Algebra, simply further including XOR gates.

$$f = \bar{A}B\bar{C}D + A\bar{B}CD + \bar{A}BC + A\bar{B}C + A\bar{B}\bar{C}D$$

[4 + 2]

d) Complete the missing entries, which are not shaded in the following table. No marks will be awarded for this question unless you show how the solution is derived.

Decimal	Hexadecimal	Signed binary (8 bits wide)	Octal
253			?
-17		?	
	?		627

[6]

e) Draw the state transition table for the finite state machine (FSM) shown in Figure 1.1.

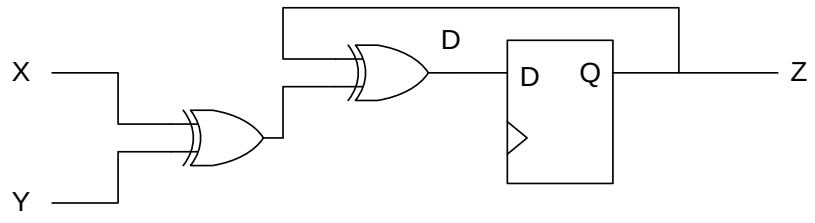


Figure 1.1

[4]

f) Determine the number of bits needed to represent the range of results obtained when two *N*-bit signed numbers are multiplied together. Your answer should explain your arguments.

[4]

g) Consider the Boolean function:

$$f = \bar{A}\,\bar{B} + AB + \bar{A}C$$

i) Implement this function using one 4 × 1 multiplexer, with the restriction that variable *A* cannot be connected to multiplexer select lines.

[4]

ii) Implement function *f* again, this time using two 2 × 1 multiplexers.

[4]

2. A finite state machine (FSM), with two inputs X1, X2, and one output Z, is shown in Figure 2.1. The FSM is initially reset to the state with Z = 0. The output of the FSM does not change unless one of the following sequences occurs at the input terminals:

i) The input sequence X1X2 = 01, followed by X1X2 = 10, causing the output to become 0.
ii) The input sequence X1X2 = 11, followed by X1X2 = 10, causing the output to become 1.
iii) The input sequence X1X2 = 00, followed by X1X2 = 10, causing the output to toggle.

a) The FSM is to be implemented using 4 states (S0, S1, S2, S3). Draw the Moore state diagram for this implementation.

[12]

b) The states S0 – S3 are to be encoded using a '1-hot' encoding system. This uses 4 bits (Q3 Q2 Q1 Q0) per state, as shown in the table below. Using this encoding system, derive the state transition table for the FSM.

State	Q3	Q2	Q1	Q0
S0	0	0	0	1
S1	0	0	1	0
S2	0	1	0	0
S3	1	0	0	0

[8]

c) Derive the Boolean equations for the FSM in sum-of-product form.

[10]

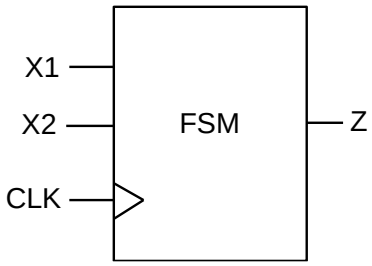


Figure 2.1

3. Figure 3.2 shows the datapath of a new CPU DECA21 comprising a register file RF1, a binary adder, and other logic. The datapath implements the ALU instructions shown in Figure 3.3. The format of the ALU instructions is shown in Figure 3.1. OP, OPC, d,m,n in Figure 3.2 refer to the unsigned binary values of bit fields OP, OPC, D, M, N in Figure 3.1. Other instructions, not shown here, are executed when OP=1. The W input to RF1 in Figure 3.2 is 1 during every cycle in which the ALU instructions are executed. W may also be 1 when other instructions are executed. Cycles with W=0 do not change any of the datapath registers.

   a) For DECA21 state, giving reasons, the number of operands of the Figure 3.3 instructions, the data word size, and number of datapath registers. Discuss whether the architecture is von Neumann, or Harvard, or could be either. [4]

   b) State two distinct types of necessary instructions not shown here that would be expected in any CPU. [2]

   c) State the largest positive and negative two's complement values in 8 bits. An 8 bit two's complement subtraction `A := B - C` is defined to *overflow* if and only if the following condition holds:

$$z(\texttt{A(7:0)}) \neq z(\texttt{B(7:0)}) - z(\texttt{C(7:0)})$$

   where $z(x)$ is the two's complement interpretation of a binary field x. State the boolean condition on the bits of `A, B, C` that computes overflow. [4]

   d) Using the instructions from Figure 3.3 write in mnemonic form a sequence of instructions that will implement `R0 := 15×R1`. Credit will be given for code that changes the minimum number of registers and uses the minimum number of instructions. [4]

   e) Write in mnemonic form a sequence of two ARM data processing instructions together implementing `R0 := 1023×R1 + 0x0850000`. For each instruction state the precise shift or rotate operation used in the ARM Op2 logic. [4]

   f) State how input busses `A1,A2,A3` in Figure 3.2 are connected to the instruction word bits `IW(21:0)`. [2]

   g) Derive a boolean expression for `FF1.EN`, from `IW(21:0)` and `W`, implementing Figure 3.3. [2]

   h) Figure 3.4 shows the logic function of block `BX`. Write a truth table implementing the instructions in Figure 3.3 with `OPC` as input and `B1, B0, MUX1.sel, MUX1.0` outputs. Outputs in your truth table which are don't care must be marked `X`. [8]

Field	OP	OPC	D	M	N
Bits used	IW21	IW(20:18)	IW(17:12)	IW(11:6)	IW(5:0)

Figure 3.1: Fields in a 22 bit Instruction Word (IW)

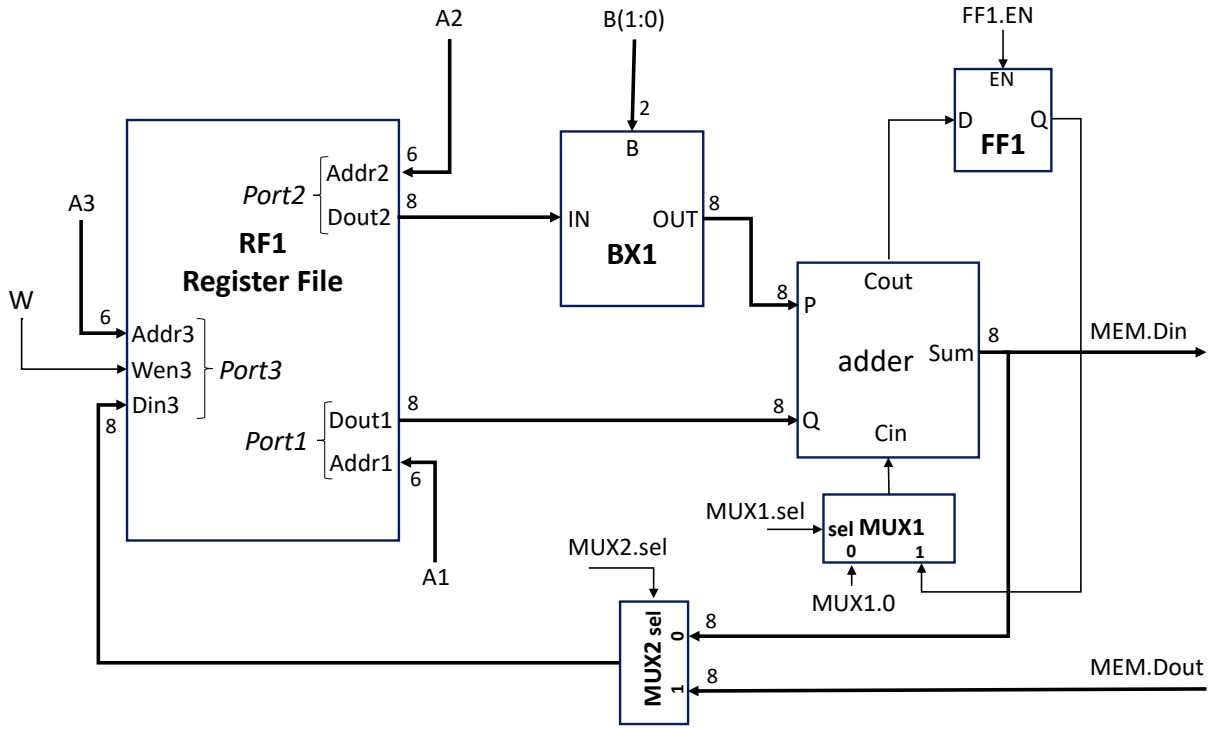


Figure 3.2: Datapath Block Diagram

Mnemonic Form	OP	OPC	Operation	C
`ADD Rd, Rn, Rm`	0	000	`Rd := Rn + Rm`	write C
`SUB Rd, Rn, Rm`	0	001	`Rd := Rn - Rm`	write C
`ADC Rd, Rn, Rm`	0	010	`Rd := Rn + Rm + C`	write C
`SBC Rd, Rn, Rm`	0	011	`Rd := Rn - Rm + C - 1`	write C
`MOV Rd, Rn`	0	100	`Rd := Rn`	hold C
`MOVC Rd, Rn`	0	101	`Rd := Rn + C`	write C
`MOVP Rd, Rn`	0	110	`Rd := Rn + 1`	write C
`MOVN Rd, Rn`	0	111	`Rd := Rn - 1`	write C

Figure 3.3: DECA21 Instructions: C is written with carry out from the adder where shown, no other instruction writes C

B1	B0	BX.IN	BX.OUT
0	0	*a*	*a*
0	1	*a*	*a* with bits inverted
1	0	X	00000000
1	1	X	11111111

Figure 3.4: Block BX operation